

第十七课预习报告

films1.c

- 作用: 演示如何使用结构体数组存储电影信息（标题和评分）。
- 功能: 程序允许用户输入最多5部电影的标题和评分，并在输入结束后输出电影列表。
- 演示效果:

```
1  /* films1.c -- using an array of structures */
2  #include <stdio.h>
3  #include <string.h>
4  #define TSIZE      45      /* size of array to hold title */
5  #define FMAX       5       /* maximum number of film titles */
6
7  struct film {
8      char title[TSIZE];
9      int rating;
10 };
11 char * s_gets(char str[], int lim);
12 int main(void)
13 {
14     struct film movies[FMAX];
15     int i = 0;
16     int j;
17
18     puts("Enter first movie title:");
19     while (i < FMAX && s_gets(movies[i].title, TSIZE) != NULL &&
20         movies[i].title[0] != '\0')
21     {
22         puts("Enter your rating <0-10>:");
23         scanf("%d", &movies[i++].rating);
24         while(getchar() != '\n')
25             continue;
26         puts("Enter next movie title (empty line to stop):");
27     }
28     if (i == 0)
29         printf("No data entered. ");
30     else
31         printf("Here is the movie list:\n");
```

```

32
33     for (j = 0; j < i; j++)
34         printf("Movie: %s Rating: %d\n", movies[j].title,
35             movies[j].rating);
36     printf("Bye!\n");
37
38     return 0;
39 }
40
41 char * s_gets(char * st, int n)
42 {
43     char * ret_val;
44     char * find;
45
46     ret_val = fgets(st, n, stdin);
47     if (ret_val)
48     {
49         find = strchr(st, '\n');    // look for newline
50         if (find)                   // if the address is not NULL,
51             *find = '\0';           // place a null character there
52         else
53             while (getchar() != '\n')
54                 continue;           // dispose of rest of line
55     }
56     return ret_val;
57 }

```

```

1  Enter first movie title:
2  Enter your rating <0-10>:
3  Enter next movie title (empty line to stop):
4  Enter your rating <0-10>:
5  Enter next movie title (empty line to stop):
6  Here is the movie list:
7  Movie: Inception Rating: 9
8  Movie: Interstellar Rating: 10
9  Bye!

```

films2.c

- 作用: 演示如何使用链表存储电影信息 (标题和评分)。
- 功能: 程序允许用户输入电影标题和评分, 使用动态分配的结构体节点形成链表, 并在输入结束后

输出电影列表，最后释放内存。

- 演示效果:

```
1  /* films2.c -- using a linked list of structures */
2  #include <stdio.h>
3  #include <stdlib.h>      /* has the malloc prototype      */
4  #include <string.h>      /* has the strcpy prototype  */
5  #define TSIZE    45     /* size of array to hold title */
6
7  struct film {
8      char title[TSIZE];
9      int rating;
10     struct film * next; /* points to next struct in list */
11 };
12 char * s_gets(char * st, int n);
13 int main(void)
14 {
15     struct film * head = NULL;
16     struct film * prev, * current;
17     char input[TSIZE];
18
19     /* Gather and store information */
20     puts("Enter first movie title:");
21     while (s_gets(input, TSIZE) != NULL && input[0] != '\0')
22     {
23         current = (struct film *) malloc(sizeof(struct film));
24         if (head == NULL) /* first structure */
25             head = current;
26         else /* subsequent structures */
27             prev->next = current;
28         current->next = NULL;
29         strcpy(current->title, input);
30         puts("Enter your rating <0-10>:");
31         scanf("%d", &current->rating);
32         while(getchar() != '\n')
33             continue;
34         puts("Enter next movie title (empty line to stop):");
35         prev = current;
36     }
37
38     /* Show list of movies */
39     if (head == NULL)
40         printf("No data entered. ");
41     else
```

```

42     printf ("Here is the movie list:\n");
43     current = head;
44     while (current != NULL)
45     {
46         printf("Movie: %s  Rating: %d\n",
47             current→title, current→rating);
48         current = current→next;
49     }
50
51     /* Program done, so free allocated memory */
52     current = head;
53     while (current != NULL)
54     {
55         free(current);
56         current = current→next;
57     }
58     printf("Bye!\n");
59
60     return 0;
61 }
62
63 char * s_gets(char * st, int n)
64 {
65     char * ret_val;
66     char * find;
67
68     ret_val = fgets(st, n, stdin);
69     if (ret_val)
70     {
71         find = strchr(st, '\n');    // look for newline
72         if (find)                    // if the address is not NULL,
73             *find = '\0';           // place a null character there
74         else
75             while (getchar() != '\n')
76                 continue;          // dispose of rest of line
77     }
78     return ret_val;
79 }

```

```
1 Enter first movie title:
2 Enter your rating <0-10>:
3 Enter next movie title (empty line to stop):
4 Enter your rating <0-10>:
5 Enter next movie title (empty line to stop):
6 Here is the movie list:
7 Movie: Inception Rating: 9
8 Movie: Interstellar Rating: 10
9 Bye!
```

list.h

- 作用: 定义链表的抽象数据类型 (ADT) 接口和数据结构。
- 功能: 提供电影信息存储的结构体 `film` , 节点结构 `node` 以及链表类型 `List` , 并声明了链表的基本操作函数原型, 供 `list.c` 和其他文件调用。
- 演示效果:

```
1  /* list.h -- header file for a simple list type */
2  #ifndef LIST_H_
3  #define LIST_H_
4  #include <stdbool.h>      /* C99 feature      */
5
6  /* program-specific declarations */
7
8  #define TSIZE      45    /* size of array to hold title */
9  struct film
10 {
11     char title[TSIZE];
12     int rating;
13 };
14
15 /* general type definitions */
16
17 typedef struct film Item;
18
19 typedef struct node
20 {
21     Item item;
22     struct node * next;
23 } Node;
24
```

```

25  typedef Node * List;
26
27  /* function prototypes */
28
29  /* operation:          initialize a list                      */
30  /* preconditions:      plist points to a list                */
31  /* postconditions:     the list is initialized to empty      */
32  void InitializeList(List * plist);
33
34  /* operation:          determine if list is empty            */
35  /*                    plist points to an initialized list    */
36  /* postconditions:     function returns True if list is empty */
37  /*                    and returns False otherwise           */
38  bool ListIsEmpty(const List *plist);
39
40  /* operation:          determine if list is full              */
41  /*                    plist points to an initialized list    */
42  /* postconditions:     function returns True if list is full  */
43  /*                    and returns False otherwise           */
44  bool ListIsFull(const List *plist);
45
46  /* operation:          determine number of items in list     */
47  /*                    plist points to an initialized list    */
48  /* postconditions:     function returns number of items in list */
49  unsigned int ListItemCount(const List *plist);
50
51  /* operation:          add item to end of list                */
52  /* preconditions:      item is an item to be added to list   */
53  /*                    plist points to an initialized list    */
54  /* postconditions:     if possible, function adds item to end */
55  /*                    of list and returns True; otherwise the */
56  /*                    function returns False                  */
57  bool AddItem(Item item, List * plist);
58
59  /* operation:          apply a function to each item in list */
60  /*                    plist points to an initialized list    */
61  /*                    pfun points to a function that takes an */
62  /*                    Item argument and has no return value   */
63  /* postcondition:      the function pointed to by pfun is     */
64  /*                    executed once for each item in the list */
65  void Traverse (const List *plist, void (* pfun)(Item item) );
66
67  /* operation:          free allocated memory, if any         */
68  /*                    plist points to an initialized list    */

```

```

69  /* postconditions:  any memory allocated for the list is freed */
70  /*
71  and the list is set to empty
72  */
73  void EmptyTheList(List * plist);
74  #endif

```

```

1  # 头文件无独立运行效果，编译通过：
2  gcc -c list.c

```

list.c

- 作用: 提供链表操作的支持函数，实现初始化、判空、判满、计数、添加节点、遍历和释放内存等功能。
- 功能: 该文件定义了链表的基本操作接口，供其他程序（如 `films3.c`）调用，实现抽象数据类型的链表管理。
- 演示效果:

```

1  /* list.c -- functions supporting list operations */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include "list.h"

```

```

1  # 该文件为库文件，无独立运行效果，编译通过：
2  gcc list.c -c

```

tree.c

- 作用: 提供二叉查找树（Binary Search Tree, BST）的实现，用于管理宠物信息的插入、查找、删除和遍历等操作。
- 功能: 与 `petclub.c` 配合使用，定义树节点结构、初始化函数、节点添加、删除、查找、统计及遍历操作，实现宠物俱乐部信息的高效管理。
- 演示效果:

```

1  /* tree.c -- tree support functions */
2  #include <string.h>
3  #include <stdio.h>
4  #include <stdlib.h>

```

```

5  #include "tree.h"
6
7  typedef struct pair {
8      Node * parent;
9      Node * child;
10 } Pair;
11
12 static Node * MakeNode(const Item * pi);
13 static bool ToLeft(const Item * i1, const Item * i2);
14 static bool ToRight(const Item * i1, const Item * i2);
15 static void AddNode (Node * new_node, Node * root);
16 static void InOrder(const Node * root, void (* pfun)(Item item));
17 static Pair SeekItem(const Item * pi, const Tree * ptree);
18 static void DeleteNode(Node ** ptr);
19 static void DeleteAllNodes(Node * ptr);
20
21 void InitializeTree(Tree * ptree)
22 {
23     ptree→root = NULL;
24     ptree→size = 0;
25 }
26
27 bool TreeIsEmpty(const Tree * ptree)
28 {
29     return ptree→root == NULL;
30 }
31
32 bool TreeIsFull(const Tree * ptree)
33 {
34     return ptree→size ≥ MAXITEMS;
35 }
36
37 int TreeItemCount(const Tree * ptree)
38 {
39     return ptree→size;
40 }
41
42 bool AddItem(const Item * pi, Tree * ptree)
43 {
44     Node * new_node;
45     if (TreeIsFull(ptree))
46     {
47         fprintf(stderr, "Tree is full\n");
48         return false;

```



```

49     }
50     new_node = MakeNode(pi);
51     if (new_node == NULL)
52     {
53         fprintf(stderr, "Couldn't create node\n");
54         return false;
55     }
56     if (ptree->root == NULL)
57         ptree->root = new_node;
58     else
59         AddNode(new_node, ptree->root);
60     ptree->size++;
61     return true;
62 }
63
64 bool InTree(const Item * pi, const Tree * ptree)
65 {
66     return SeekItem(pi, ptree).child != NULL;
67 }
68
69 bool DeleteItem(const Item * pi, Tree * ptree)
70 {
71     Pair look;
72     look = SeekItem(pi, ptree);
73     if (look.child == NULL)
74         return false;
75     if (look.parent == NULL)
76         DeleteNode(&ptree->root);
77     else if (look.parent->left == look.child)
78         DeleteNode(&look.parent->left);
79     else
80         DeleteNode(&look.parent->right);
81     ptree->size--;
82     return true;
83 }
84
85 void Traverse (const Tree * ptree, void (* pfun)(Item item))
86 {
87     if (ptree != NULL)
88         InOrder(ptree->root, pfun);
89 }
90
91 void DeleteAll(Tree * ptree)
92 {

```

```
93     if (ptree != NULL)
94         DeleteAllNodes(ptree->root);
95     ptree->root = NULL;
96     ptree->size = 0;
97 }
```

```
1  # tree.c 为库文件, 与 petclub.c 配合使用, 单独编译验证通过:
2  gcc -c tree.c
```

```
/* local function prototype */
static void CopyToNode(Item item, Node * pnode);
```

```
/* interface functions /
/ set the list to empty */
void InitializeList(List * plist)
{
    * plist = NULL;
}
```

```
/* returns true if list is empty */
bool ListIsEmpty(const List * plist)
{
    if (*plist == NULL)
        return true;
    else
        return false;
}
```

```
/* returns true if list is full */
bool ListIsFull(const List * plist)
{
    Node * pt;
    bool full;
```

```

1 | pt = (Node *) malloc(sizeof(Node));
2 | if (pt == NULL)
3 |     full = true;
4 | else
5 |     full = false;
6 | free(pt);
7 |
8 | return full;

```

```

}

```

/ returns number of nodes */*

unsigned int ListItemCount(const List * plist)

```

{
    unsigned int count = 0;
    Node * pnode = plist; / set to start of list */

```

```

1 | while (pnode != NULL)
2 | {
3 |     ++count;
4 |     pnode = pnode->next; / set to next node */
5 | }
6 |
7 | return count;

```

```

}

```

/ creates node to hold item and adds it to the end of /*

*/ the list pointed to by plist (slow implementation) */*

bool AddItem(Item item, List * plist)

```

{
    Node * pnew;
    Node * scan = *plist;

```

```

1 | pnew = (Node *) malloc(sizeof(Node));
2 | if (pnew == NULL)
3 |     return false; / quit function on failure */
4 |
5 | CopyToNode(item, pnew);

```

```

6   pnew→next = NULL;
7   if (scan == NULL)           /* empty list, so place */
8       *plist = pnew;          /* pnew at head of list */
9   else
10  {
11      while (scan→next ≠ NULL)
12          scan = scan→next; /* find end of list */
13      scan→next = pnew;      /* add pnew to end */
14  }
15
16  return true;

```

```

}

```

/* visit each node and execute function pointed to by pfun */

void Traverse (const List * plist, void (* pfun)(Item item))

```

{

```

Node * pnode = *plist*; / set to start of list */

```

1   while (pnode ≠ NULL)
2   {
3       (*pfun)(pnode→item); /* apply function to item */
4       pnode = pnode→next; /* advance to next item */
5   }

```

```

}

```

/* free memory allocated by malloc() /

/ set list pointer to NULL */

void EmptyTheList(List * plist)

```

{

```

Node * psave;

```

1   while (*plist ≠ NULL)
2   {
3       psave = (*plist)→next; /* save address of next node */
4       free(*plist);          /* free current node */
5       *plist = psave;        /* advance to next node */
6   }

```

```

}

/* local function definition /
/ copies an item into a node */
static void CopyToNode(Item item, Node * pnode)
{
    pnode->item = item; /* structure copy */
}

```

```

1  `` `bash
2  # 该文件为库文件，无独立运行效果，编译通过：
3  gcc list.c -c

```

films3.c

- 作用: 演示如何使用抽象数据类型（ADT）方式的链表管理电影信息。
- 功能: 程序通过 `list.h` 定义的接口创建、操作、遍历及释放链表节点，动态存储电影标题及评分，最终统计电影总数。
- 演示效果:

```

1  /* films3.c -- using an ADT-style linked list */
2  /* compile with list.c */
3  #include <stdio.h>
4  #include <stdlib.h> /* prototype for exit() */
5  #include "list.h" /* defines List, Item */
6  #include <string.h> /* for strchr */
7  void showmovies(Item item);
8  char * s_gets(char * st, int n);
9  int main(void)
10 {
11     List movies;
12     Item temp;
13
14
15     /* initialize */
16     InitializeList(&movies);
17     if (ListIsFull(&movies))
18     {
19         fprintf(stderr, "No memory available! Bye!\n");
20         exit(1);
21     }

```

```

22
23     /* gather and store */
24     puts("Enter first movie title:");
25     while (s_gets(temp.title, TSIZE) != NULL && temp.title[0] != '\0')
26     {
27         puts("Enter your rating <0-10>:");
28         scanf("%d", &temp.rating);
29         while(getchar() != '\n')
30             continue;
31         if (AddItem(temp, &movies)==false)
32         {
33             fprintf(stderr, "Problem allocating memory\n");
34             break;
35         }
36         if (ListIsFull(&movies))
37         {
38             puts("The list is now full.");
39             break;
40         }
41         puts("Enter next movie title (empty line to stop):");
42     }
43
44     /* display          */
45     if (ListIsEmpty(&movies))
46         printf("No data entered. ");
47     else
48     {
49         printf ("Here is the movie list:\n");
50         Traverse(&movies, showmovies);
51     }
52     printf("You entered %d movies.\n", ListItemCount(&movies));
53
54
55     /* clean up        */
56     EmptyTheList(&movies);
57     printf("Bye!\n");
58
59     return 0;
60 }
61
62 void showmovies(Item item)
63 {
64     printf("Movie: %s   Rating: %d\n", item.title,
65         item.rating);

```

```

66     }
67
68     char * s_gets(char * st, int n)
69     {
70         char * ret_val;
71         char * find;
72
73         ret_val = fgets(st, n, stdin);
74         if (ret_val)
75         {
76             find = strchr(st, '\n');    // look for newline
77             if (find)                    // if the address is not NULL,
78                 *find = '\0';           // place a null character there
79             else
80                 while (getchar() != '\n')
81                     continue;           // dispose of rest of line
82         }
83         return ret_val;
84     }

```

```

1   Enter first movie title:
2   Enter your rating <0-10>:
3   Enter next movie title (empty line to stop):
4   Enter your rating <0-10>:
5   Enter next movie title (empty line to stop):
6   Here is the movie list:
7   Movie: Inception   Rating: 9
8   Movie: Interstellar Rating: 10
9   You entered 2 movies.
10  Bye!

```

mall.c

- 作用: 使用队列接口模拟顾客排队咨询的过程。
- 功能: 程序通过随机生成顾客到达时间和咨询时长, 统计在给定模拟时间内的顾客接受数、服务数、拒绝数、平均队列长度和平均等待时间。
- 演示效果:

```

1   // mall.c -- use the Queue interface
2   // compile with queue.c
3   #include <stdio.h>

```

```

4  #include <stdlib.h>    // for rand() and srand()
5  #include <time.h>      // for time()
6  #include "queue.h"     // change Item typedef
7  #define MIN_PER_HR 60.0
8
9  bool newcustomer(double x); // is there a new customer?
10 Item customertime(long when); // set customer parameters
11
12 int main(void)
13 {
14     Queue line;
15     Item temp;           // new customer data
16     int hours;           // hours of simulation
17     int perhour;         // average # of arrivals per hour
18     long cycle, cyclelimit; // loop counter, limit
19     long turnaways = 0;  // turned away by full queue
20     long customers = 0;  // joined the queue
21     long served = 0;     // served during the simulation
22     long sum_line = 0;   // cumulative line length
23     int wait_time = 0;   // time until Sigmund is free
24     double min_per_cust; // average time between arrivals
25     long line_wait = 0;  // cumulative time in line
26
27     InitializeQueue(&line);
28     srand((unsigned int) time(0)); // random initializing of rand()
29     puts("Case Study: Sigmund Lander's Advice Booth");
30     puts("Enter the number of simulation hours:");
31     scanf("%d", &hours);
32     cyclelimit = MIN_PER_HR * hours;
33     puts("Enter the average number of customers per hour:");
34     scanf("%d", &perhour);
35     min_per_cust = MIN_PER_HR / perhour;
36
37     for (cycle = 0; cycle < cyclelimit; cycle++)
38     {
39         if (newcustomer(min_per_cust))
40         {
41             if (QueueIsFull(&line))
42                 turnaways++;
43             else
44             {
45                 customers++;
46                 temp = customertime(cycle);
47                 EnQueue(temp, &line);

```



```

48         }
49     }
50     if (wait_time ≤ 0 && !QueueIsEmpty(&line))
51     {
52         DeQueue (&temp, &line);
53         wait_time = temp.processtime;
54         line_wait += cycle - temp.arrive;
55         served++;
56     }
57     if (wait_time > 0)
58         wait_time--;
59     sum_line += QueueItemCount(&line);
60 }
61
62 if (customers > 0)
63 {
64     printf("customers accepted: %ld\n", customers);
65     printf("  customers served: %ld\n", served);
66     printf("      turnaways: %ld\n", turnaways);
67     printf("average queue size: %.2f\n",
68           (double) sum_line / cyclelimit);
69     printf(" average wait time: %.2f minutes\n",
70           (double) line_wait / served);
71 }
72 else
73     puts("No customers!");
74 EmptyTheQueue(&line);
75 puts("Bye!");
76
77 return 0;
78 }

```

```

1  Case Study: Sigmund Lander's Advice Booth
2  Enter the number of simulation hours:
3  Enter the average number of customers per hour:
4  customers accepted: 18
5      customers served: 18
6          turnaways: 0
7  average queue size: 0.02
8  average wait time: 0.11 minutes
9  Bye!

```

petclub.c

- 作用: 使用二叉查找树存储和管理宠物会员信息。
- 功能: 提供添加宠物、显示所有宠物、统计数量、查找宠物、删除宠物等功能, 通过 `tree.c` 实现树结构管理。
- 演示效果:

```
1  /* petclub.c -- use a binary search tree */
2  #include <stdio.h>
3  #include <string.h>
4  #include <ctype.h>
5  #include "tree.h"
6
7  char menu(void);
8  void addpet(Tree * pt);
9  void droppet(Tree * pt);
10 void showpets(const Tree * pt);
11 void findpet(const Tree * pt);
12 void printitem(Item item);
13 void uppercase(char * str);
14 char * s_gets(char * st, int n);
15
16 int main(void)
17 {
18     Tree pets;
19     char choice;
20
21     InitializeTree(&pets);
22     while ((choice = menu()) != 'q')
23     {
24         switch (choice)
25         {
26             case 'a' : addpet(&pets);
27                         break;
28             case 'l' : showpets(&pets);
29                         break;
30             case 'f' : findpet(&pets);
31                         break;
32             case 'n' : printf("%d pets in club\n",
33                               TreeItemCount(&pets));
34                         break;
35             case 'd' : droppet(&pets);
```

```

36         break;
37         default : puts("Switching error");
38     }
39 }
40 DeleteAll(&pets);
41 puts("Bye.");
42
43 return 0;
44 }

```

```

1  Nerfville Pet Club Membership Program
2  Enter the letter corresponding to your choice:
3  a) add a pet          l) show list of pets
4  n) number of pets    f) find pets
5  d) delete a pet      q) quit
6  Bye.

```

use_q.c

- 作用: 验证队列接口的基本功能。
- 功能: 程序测试 `queue.c` 中的函数, 通过交互方式执行添加、删除操作, 并显示队列项数量的变化情况。
- 演示效果:

```

1  Testing the Queue interface. Type a to add a value,
2  type d to delete a value, and type q to quit.
3  Integer to add: Putting 5 into queue
4  1 items in queue
5  Type a to add, d to delete, q to quit:
6  Integer to add: Putting 8 into queue
7  2 items in queue
8  Type a to add, d to delete, q to quit:
9  Removing 5 from queue
10  1 items in queue
11  Type a to add, d to delete, q to quit:
12  Bye!

```

- 作用: 使用二叉查找树存储和管理宠物会员信息。
- 功能: 提供添加宠物、显示所有宠物、统计数量、查找宠物、删除宠物等功能, 通过 `tree.c` 实现

树结构管理。

- 演示效果:

```
1  Nerfville Pet Club Membership Program
2  Enter the letter corresponding to your choice:
3  a) add a pet          l) show list of pets
4  n) number of pets    f) find pets
5  d) delete a pet      q) quit
6  Bye.
```

- 作用: 使用队列接口模拟顾客排队咨询的过程。
- 功能: 程序通过随机生成顾客到达时间和咨询时长, 统计在给定模拟时间内的顾客接受数、服务数、拒绝数、平均队列长度和平均等待时间。
- 演示效果:

```
1  // mall.c -- use the Queue interface
2  // compile with queue.c
3  #include <stdio.h>
4  #include <stdlib.h>    // for rand() and srand()
5  #include <time.h>      // for time()
6  #include "queue.h"     // change Item typedef
7  #define MIN_PER_HR 60.0
8
9  bool newcustomer(double x); // is there a new customer?
10 Item customertime(long when); // set customer parameters
11
12 int main(void)
13 {
14     Queue line;
15     Item temp;           // new customer data
16     int hours;           // hours of simulation
17     int perhour;         // average # of arrivals per hour
18     long cycle, cyclelimit; // loop counter, limit
19     long turnaways = 0;  // turned away by full queue
20     long customers = 0;  // joined the queue
21     long served = 0;     // served during the simulation
22     long sum_line = 0;   // cumulative line length
23     int wait_time = 0;   // time until Sigmund is free
24     double min_per_cust; // average time between arrivals
25     long line_wait = 0;  // cumulative time in line
26
27     InitializeQueue(&line);
```

```

28     srand((unsigned int) time(0)); // random initializing of rand()
29     puts("Case Study: Sigmund Lander's Advice Booth");
30     puts("Enter the number of simulation hours:");
31     scanf("%d", &hours);
32     cyclelimit = MIN_PER_HR * hours;
33     puts("Enter the average number of customers per hour:");
34     scanf("%d", &perhour);
35     min_per_cust = MIN_PER_HR / perhour;
36
37     for (cycle = 0; cycle < cyclelimit; cycle++)
38     {
39         if (newcustomer(min_per_cust))
40         {
41             if (QueueIsFull(&line))
42                 turnaways++;
43             else
44             {
45                 customers++;
46                 temp = customertime(cycle);
47                 EnQueue(temp, &line);
48             }
49         }
50         if (wait_time ≤ 0 && !QueueIsEmpty(&line))
51         {
52             DeQueue (&temp, &line);
53             wait_time = temp.processtime;
54             line_wait += cycle - temp.arrive;
55             served++;
56         }
57         if (wait_time > 0)
58             wait_time--;
59         sum_line += QueueItemCount(&line);
60     }
61
62     if (customers > 0)
63     {
64         printf("customers accepted: %ld\n", customers);
65         printf("  customers served: %ld\n", served);
66         printf("      turnaways: %ld\n", turnaways);
67         printf("average queue size: %.2f\n",
68             (double) sum_line / cyclelimit);
69         printf(" average wait time: %.2f minutes\n",
70             (double) line_wait / served);
71     }

```

```

72     else
73         puts("No customers!");
74     EmptyTheQueue(&line);
75     puts("Bye!");
76
77     return 0;
78 }

```

```

1 Case Study: Sigmund Lander's Advice Booth
2 Enter the number of simulation hours:
3 Enter the average number of customers per hour:
4 customers accepted: 18
5     customers served: 18
6         turnaways: 0
7 average queue size: 0.02
8     average wait time: 0.11 minutes
9 Bye!

```

- 作用: 演示如何使用抽象数据类型 (ADT) 方式的链表管理电影信息。
- 功能: 程序通过 `list.h` 定义的接口创建、操作、遍历及释放链表节点, 动态存储电影标题及评分, 最终统计电影总数。
- 主要错误: 缺少 `#include <string.h>` 导致 `strchr` 未声明, 已修复。
- 演示效果:

```

1  /* films3.c -- using an ADT-style linked list */
2  /* compile with list.c */
3  #include <stdio.h>
4  #include <stdlib.h> /* prototype for exit() */
5  #include "list.h" /* defines List, Item */
6  #include <string.h> /* for strchr */
7  void showmovies(Item item);
8  char * s_gets(char * st, int n);
9  int main(void)
10 {
11     List movies;
12     Item temp;
13
14
15     /* initialize */
16     InitializeList(&movies);
17     if (ListIsFull(&movies))

```

```

18     {
19         fprintf(stderr, "No memory available! Bye!\n");
20         exit(1);
21     }
22
23     /* gather and store */
24     puts("Enter first movie title:");
25     while (s_gets(temp.title, TSIZE) != NULL && temp.title[0] != '\0')
26     {
27         puts("Enter your rating <0-10>:");
28         scanf("%d", &temp.rating);
29         while(getchar() != '\n')
30             continue;
31         if (AddItem(temp, &movies)==false)
32         {
33             fprintf(stderr, "Problem allocating memory\n");
34             break;
35         }
36         if (ListIsFull(&movies))
37         {
38             puts("The list is now full.");
39             break;
40         }
41         puts("Enter next movie title (empty line to stop):");
42     }
43
44     /* display      */
45     if (ListIsEmpty(&movies))
46         printf("No data entered. ");
47     else
48     {
49         printf ("Here is the movie list:\n");
50         Traverse(&movies, showmovies);
51     }
52     printf("You entered %d movies.\n", ListItemCount(&movies));
53
54
55     /* clean up      */
56     EmptyTheList(&movies);
57     printf("Bye!\n");
58
59     return 0;
60 }
61

```

```

62 void showmovies(Item item)
63 {
64     printf("Movie: %s Rating: %d\n", item.title,
65           item.rating);
66 }
67
68 char * s_gets(char * st, int n)
69 {
70     char * ret_val;
71     char * find;
72
73     ret_val = fgets(st, n, stdin);
74     if (ret_val)
75     {
76         find = strchr(st, '\n'); // look for newline
77         if (find)                // if the address is not NULL,
78             *find = '\0';        // place a null character there
79     else
80         while (getchar() != '\n')
81             continue;           // dispose of rest of line
82     }
83     return ret_val;
84 }

```

```

1  Enter first movie title:
2  Enter your rating <0-10>:
3  Enter next movie title (empty line to stop):
4  Enter your rating <0-10>:
5  Enter next movie title (empty line to stop):
6  Here is the movie list:
7  Movie: Inception Rating: 9
8  Movie: Interstellar Rating: 10
9  You entered 2 movies.
10 Bye!

```